Ranking and price valuation comparison

Your Ptixiaki model versus the attached nineframes-eiv-worker archive. Focus: how prices are trained, predicted, and affected by geographic information. Static source review; no retraining or database changes.

Answer: the pipelines have a similar two-tree shape, but their price targets, training populations, geographic premiums, and feature contracts differ enough that similar property estimates cannot be assumed.

What is genuinely similar

• Both fit a first XGBoost model, calculate (target - prediction) / target × 100 as the first-stage residual, average nearby residuals as GeoPremium, then fit a second XGBoost model that includes GeoPremium.

• Both configure histogram trees with depth 8, minimum child weight 30, 500 estimators, learning rate 0.05, and 0.8 row/column sampling. Both export a second-stage ONNX model.

Differences at a glance

Factor	Your project	Archive ranking
What tree learns	Listing asking EUR/m²	Adjusted market-like EUR/m²; log1p target
Training data	Scraped listings meeting project filters	Different SQL filters, valuation-row requirement, plus eligible verified sales for sell
Spatial input	Up to 50 residual neighbors within 300 m	Up to 50 within 160 m; new-property fallback copies a saved nearby premium
Residence features	Engineered 30-field phase-two contract	Raw DB fields for training; engineered Go fields for inference
Total price	Predicted EUR/m² × area	expm1(model output) × area, truncated to integer

How to read this report. A reported difference in model design does not predict a fixed euro gap or a consistent direction. The archive has no trained price artifacts in the ZIP, so this is a code-level comparison, not a matched-property backtest.

1. The trees optimize different price targets

Your trainer fits AskingPricePerSqm directly. If a listing asks €240,000 for 100 m², its label is €2,400/m². The archive first transforms its label with log1p; its Go predictor later uses expm1 to return EUR/m². The inverse is implemented, so this is not a units mistake. It changes the loss the trees optimize: errors at the expensive end receive relatively less weight in log space.

The archive also applies an area-specific premium correction before training: for scraped rows, adjusted asking = original asking / (1 + premium_percent / 100). Its verified government-sale rows are already treated as market values. At a +20% premium, €240,000 becomes €200,000, or €2,000/m² on a 100 m² property. This is an example of label construction, not a predicted 16.7% reduction for every home.

The correction is applied to the training label. The columns IsVerifiedSale and AreaPremiumPercent are dropped from the tree inputs. At prediction time, there is no corresponding direct division of the model output; the adjusted labels have already shaped the fitted trees.

Sale training includes a different kind of evidence

For eligible sell segments, the archive SQL adds recent, quality-filtered rows from gov_verified_sales using verified_sale_price. Your Worker training query has no such union. These rows can shift the fitted sale tree even when all tree hyperparameters match. The government rows do not enter the archive GeoPremium neighbor lookup because that lookup searches property_geo_data/properties, while the union creates synthetic gov_ UIDs; they consequently receive zero GeoPremium unless separately represented there.

A metadata error in the archive

The archive writes AskingPricePerSqm = y to phase2_predictions.json, but y is the log1p-transformed target. PredictedPricePerSqm and residuals are calculated on the original EUR/m² scale. This mislabeled field affects the saved prediction record; the reviewed direct price-prediction path uses the ONNX output and expm1, so this particular field error does not itself prove the returned estimate is logged.

2. Geographic premiums are not equivalent

Step	Your project	Archive ranking
Training neighbors	Within 300 m, nearest 50 eligible phase-one residuals	Within 160 m, nearest 50 eligible phase-one residuals
Premium	Mean of residual percentages; population spread and global z-score	Same mean and population-style z-score formula, but different neighborhood and model residuals
New property	Recalculates from up to 50 nearby stored eligible phase-one residuals	Looks for a stored premium on one of up to 25 nearest properties within 160 m; otherwise zero

A property near a neighborhood boundary or with few comparables can therefore receive a notably different GeoPremium. The two GeoPremium values also inherit differences from their first-stage models and target labels.

3. The archive has a training/inference feature mismatch

This is the largest technical obstacle to trusting close numerical results from the archive as written. Your project exports engineered property features before fitting the model and enforces an exact 30-feature order for residence/property phase two. The archive training script reads raw SQL columns, renames some columns, converts categories/numerics, and fits those columns. Its Go inference path separately builds engineered features.

Archive tree sees at training	Archive Go supplies at inference	Effect
ConstructionYear; RenovationYear	Age; RenovAge	Original fields absent from Go feature map; manifest inserts NaN
Heat; FacadeType; ViewType	HeatType; facade/view indicator flags	Raw names and representations differ; unmapped raw strings/arrays are coerced to NaN during training
AdCreationDate; entry_created_at	AdCreationAge	Timestamp fields are not matched to the engineered age field
Bathrooms; Wc	Combined WC	Training and inference values do not express the same measurement

The manifest preserves the raw training-column order. In inference, BuildFeatureArray iterates it and inserts NaN for any name absent from the Go feature map. ValidateFeatureCount checks vector length, not whether fields were found. A trained split on a raw field such as ConstructionYear can therefore follow a missing-value branch for every Go prediction. This can alter prices substantially and make them less responsive to some property attributes.

Other inputs and cohort differences

• Your residence query selects Elevator; the archive residence training query does not. Its Go residential feature struct still has Elevator, but the archive manifest cannot add a field that was never in its training query.

• Your query uses administrative boundary level 6. The archive selects the highest available level among 6, 7, 8, and 9. AdminName category granularity and learned location effects can change.

• Your supervised query requires positive p.asking_price. The archive global rule requires only non-NULL p.asking_price and tests v.asking_price > 1000 for sell rows. Zero or invalid price labels could disrupt log1p/residual calculations if present.

Interpretation: matching hyperparameters and model family are insufficient. Matching target units, rows, feature names, feature values, and spatial inputs matters more.

4. What does “needs a valuations row” mean?

In the supplied schema, valuations is a table with one record per property_uid (its primary key). It stores an asking_price, real_value, estimated_value, confidence, and optional grounded/effective valuation fields. The archive has a separate comparables/objective valuation pipeline that upserts this record. It is distinct from the archive tree result, which is stored in property_rankings.

The archive ranking training query uses an INNER JOIN valuations v ON p.uid = v.property_uid for residence, land, and commercial scraped properties. An inner join acts like a gate: a property without a matching valuations record is omitted from the training set before any tree is fitted. It is not an input feature and it does not mean the model trains on v.estimated_value.

Example property	Has valuations row?	Included by this join?
UID A; p.asking_price €200,000	Yes, matching v.property_uid = A	Yes, subject to all other training filters
UID B; p.asking_price €200,000	No	No, even if its other attributes are complete
UID C; p.asking_price €200,000; v.asking_price €900	Yes	No for sell: v.asking_price must exceed €1,000

For sell, the archive additionally requires v.asking_price > 1000. Yet its selected training price is p.asking_price, later adjusted by the area premium. If p.asking_price and v.asking_price are out of sync, the latter controls admission while the former supplies the label. For rent, the join still applies, but this €1,000 test does not. Government-sale rows in the UNION ALL are a separate path and do not require a valuations row.

Your project: its residence/property training query does not join valuations, so an otherwise eligible home is not excluded for lacking that row. Its landplot and commercial queries do join valuations, but the shown price threshold uses p.asking_price instead of v.asking_price. The archive’s normal postprocessing sequence runs valuation before ranking, which helps populate records operationally; it does not remove the SQL requirement during a training run.

Practical effect: the archive residence model can be trained on fewer and differently selected scraped homes, even when both systems read the same properties table.

What can be concluded about the numbers?

Likely material divergence, with no defensible fixed percentage. The archive optimizes a different label, uses a different sale cohort and spatial radius, and appears to pass missing values for several fields used in training. For a given property, effects may cancel or compound. The code alone cannot establish whether one implementation is systematically higher or lower across all areas and segments.

The most consequential uncertainties are the actual area premium values, how many verified-sale rows pass the archive filters, the fraction of scraped properties with valuations rows, the fitted tree splits, and the exact feature vectors delivered at inference. The ZIP contains no trained price artifacts. A fair numerical answer would require both trained models and the same held-out property set, comparing EUR/m² and total value by segment and area without retraining either production system.

Source map (line numbers in reviewed snapshot)

Your project

Worker/Models/TreeCode/Tree.py: 185-195, 671-714 (model settings, target, fit, residual). Worker/training/GeoPremium.go: 92-105, 131-210 (neighbors and premium). Worker/testing/GetGeoPremium.go: 21-83 (new-property premium). Worker/database/QueryBuilder.go: 37-61, 131-170 (training columns and eligibility). Worker/types/TypeConverter.go: 164-220 (engineered features). ml_shared/explain_artifacts.py: 25-56 (30-feature residence contract). schema_dump.sql: 1793-1803, 2390-2410, 3633-3635, 5083-5085 (properties and valuations definitions/keys).

Attached archive: nineframes-eiv-worker-main.zip

internal/ranking/training/train.py: 74-160 (DB read, area adjustment, target), 163-283 (training, predictions, manifest), 318-412 (GeoPremium). internal/ranking/configs.go: 20-63 (drops and settings). internal/ranking/rules.go: 28-57, 128-214, 244-290, 293-409 (training SQL and gov union). internal/ranking/encoder.go: 26-88 (inference encoding). internal/ranking/features.go: 42-134 (manifest-based vector and NaN fallback). internal/ranking/geopremium.go: 43-84 (online fallback). internal/ranking/service.go: 155-230 and pipeline_batched.go: 278-350 (ONNX price and total). internal/database/postgres/valuation.go: 13-35 (valuations upsert). internal/postprocessing/processor.go: 36-55 (valuation before ranking).

Scope note: This report concerns tree-based price estimates and their valuation-data dependency. It does not assess the archive’s separate priority/ranking score, nor your Elo score. No project source, model, or database records were changed.